ESTUDIAR · ENTENDER · SUSTENTAR

Catálogo y Usuario
Garritas Store

Guía explicada con C4 Model, código real y ejemplos para la exposición.

C1 Contexto · C2 Contenedores · C3 Componentes · C4 Código

Incluye Eureka, Gateway, Keycloak, Kafka, Saga, compensación y ubicaciones de credenciales de demostración.

Responsable de estos servicios: Fio.
El contenido también permite explicar cómo colaboran con Venta, Pago y Citas.

Código revisado: `cf3cede` · Rama `feat/garritas-store`
Los enlaces de código fijan esta revisión para conservar sus líneas exactas.

Ir al índice ·

Índice

1. Lo esencial para explicar en un minuto
2. Cómo leer C4 Model
3. C1: contexto de Garritas
4. C2: conexiones reales
5. Qué administra Catálogo
6. C3: componentes de Catálogo
7. C4: clases y muchos a muchos de Catálogo
8. Qué administra Usuario
9. C3: componentes de Usuario
10. C4: clases y relaciones de Usuario
11. Dónde están los CRUD y reglas
12. Eureka, Gateway y comunicación resiliente
13. Keycloak, JWT y perfil propio
14. Kafka, Outbox e Inbox
15. Saga: fallos y compensación
16. Credenciales, claves e identificadores
17. Cómo exponerlo desde Angular
18. Preguntas y respuestas rápidas
19. Mapa completo del código

> Los diagramas se pueden ampliar en el HTML. El PDF contiene texto seleccionable y enlaces al código. Las imágenes PNG y SVG también están en la carpeta `diagramas`.

1. Lo esencial para explicar en un minuto

Servicio	Qué administra	Cómo lo explicas
Catálogo	Productos, categorías, stock y reservas de inventario.	“Dice qué vendemos, cuánto cuesta y qué unidades podemos reservar”.
Usuario	Perfiles, mascotas, roles internos, planes, suscripciones y períodos.	“Organiza al cliente y registra su vínculo con los planes, conservando su historial”.

Clasificación para la sustentación

Catálogo es principalmente un servicio de datos maestros, aunque también participa transaccionalmente en la reserva y compensación del stock. No significa que carezca de transacciones.

Usuario tiene una capacidad transaccional verificable en las suscripciones: contratar, renovar y cancelar con estados e historial. El CRUD de perfiles por sí solo no explica esa capacidad.

Fio puede demostrar muchos a muchos tanto en **Producto–Categoría** como en **Usuario–Rol**. También existe Usuario–Plan mediante Suscripcion.

> No atribuyas a Usuario la emisión de tokens: el proveedor de identidad es Keycloak. No atribuyas a Usuario una Saga de Kafka: la Saga implementada de compra involucra Venta, Catálogo y Pago.

2. Cómo leer C4 Model

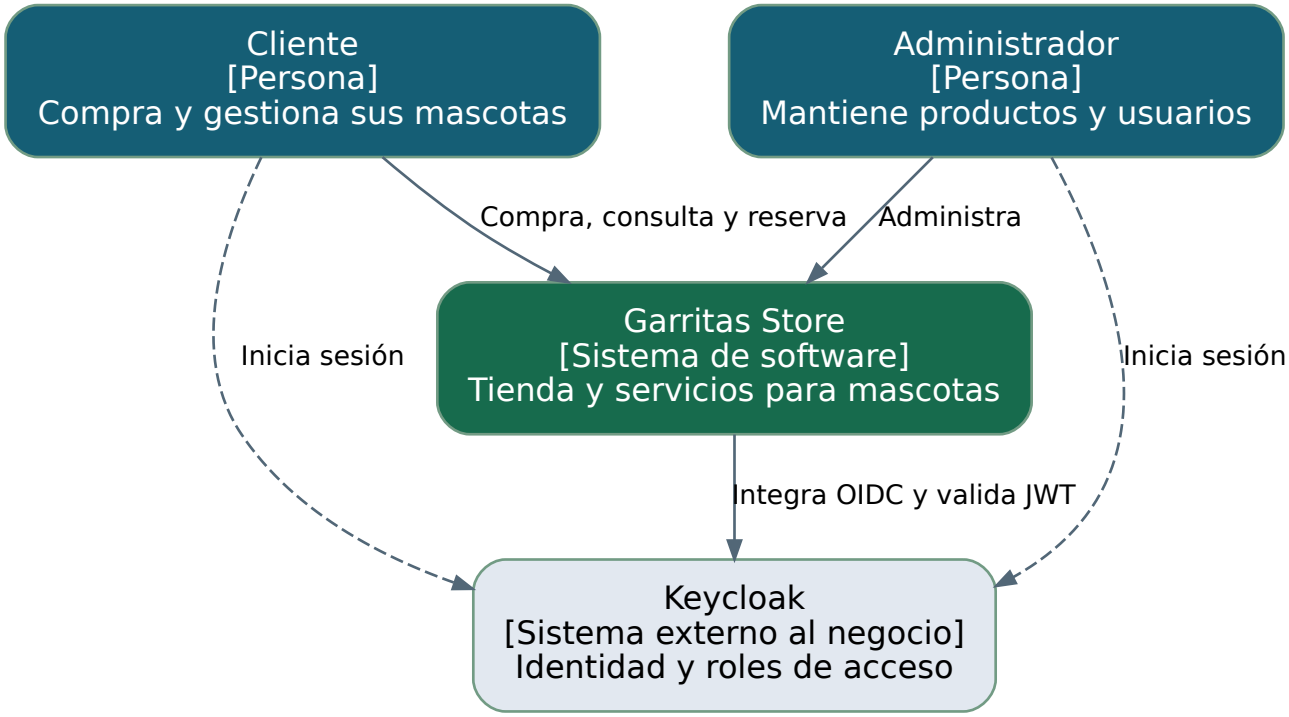
C4 es una forma de explicar una arquitectura con cuatro acercamientos. Cada nivel responde una pregunta distinta.

Nivel	Pregunta	En esta guía
C1 · Contexto	¿Quién usa el sistema y con qué sistema externo se relaciona?	Clientes, administradores, Garritas y Keycloak.
C2 · Contenedores	¿Qué aplicaciones y almacenes ejecutables lo componen?	Angular, Gateway, microservicios, Kafka y PostgreSQL.
C3 · Componentes	¿Cómo se organiza por dentro una aplicación?	Controllers, services, repositorios y mensajería.
C4 · Código	¿Qué clases y relaciones implementan un componente?	Entidades, asociaciones y métodos reales de Catálogo y Usuario.

“Contenedor” en C4 no significa obligatoriamente contenedor Docker. Un microservicio ejecutado como proceso Java sigue siendo un contenedor de software en el modelo C4.

Los diagramas de nivel 4 usan clases reales y anotaciones JPA. Cuando aparece una tabla SQL como apoyo, se identifica explícitamente como tabla, no como clase Java.

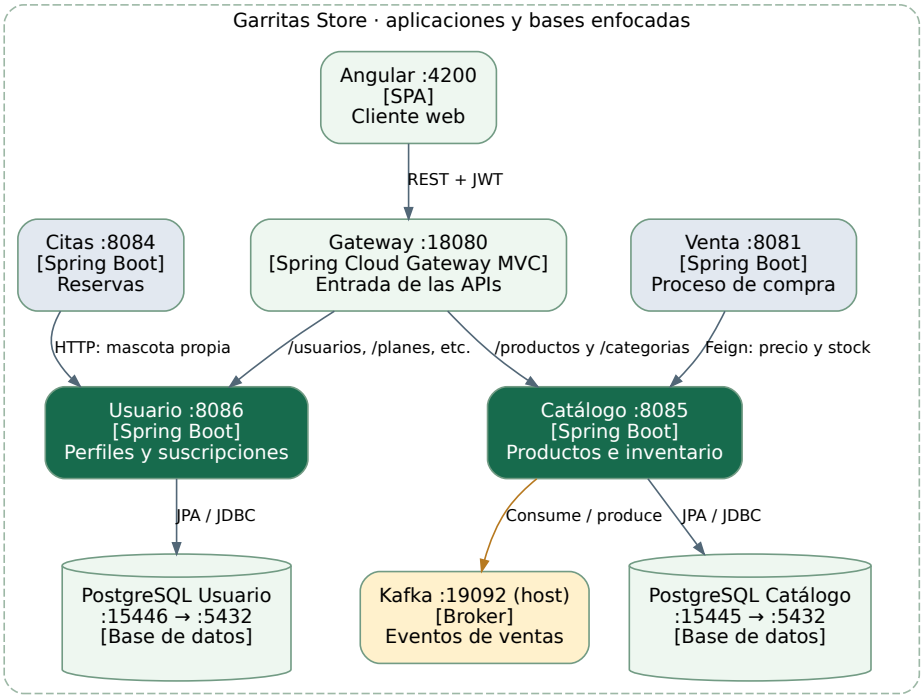
3. C1: contexto de Garritas



C4 · Nivel 1 — Contexto del sistema

El cliente compra y gestiona a sus mascotas. El administrador mantiene información y opera la tienda. Keycloak proporciona autenticación y roles de acceso; forma parte de la solución desplegada, pero está fuera de la lógica propia del negocio.

4. C2: conexiones reales



C4 · Nivel 2 — Contenedores: foco en Catálogo y Usuario

Esta es una vista enfocada: también existen Pago, Institución, Envío y Bienestar, además de Eureka, Keycloak, Config Server y observabilidad. Venta y Pago también intercambian eventos con Kafka, como muestra el diagrama de Saga. No se dibujan todos sus enlaces para mantener legible el acercamiento a los dos servicios estudiados.

Catálogo y Usuario **no comparten base de datos ni necesitan llamarse directamente** para sus CRUD. Venta consulta Catálogo por Feign. Citas consulta Usuario para verificar la mascota del cliente. Las llamadas del navegador a las APIs de negocio pasan por Gateway; el login se comunica con Keycloak.

Componente DEV	Puerto de tu PC	Función
Angular	4200	Interfaz
Gateway	18080	Entrada de APIs
Catálogo / PostgreSQL	8085 / 15445	Productos e inventario
Usuario / PostgreSQL	8086 / 15446	Perfiles y suscripciones
Eureka / Config Server	18761 / 18888	Descubrimiento / configuración
Keycloak / Kafka UI	18180 / 18085	Identidad / inspección de eventos
Kafka / Grafana / Prometheus	19092 / 13000 / 19090	Mensajería / visualización / métricas

5. Qué administra Catálogo

Responsabilidades

- **Producto:** nombre, descripción, precio, stock, estado y categorías.
- **Categoría:** clasificación reutilizable: alimentos, higiene, juguetes, etc.
- **Reserva de inventario:** descontar disponibilidad de manera controlada para una venta.
- **Compensación:** liberar la reserva si el pago es rechazado.

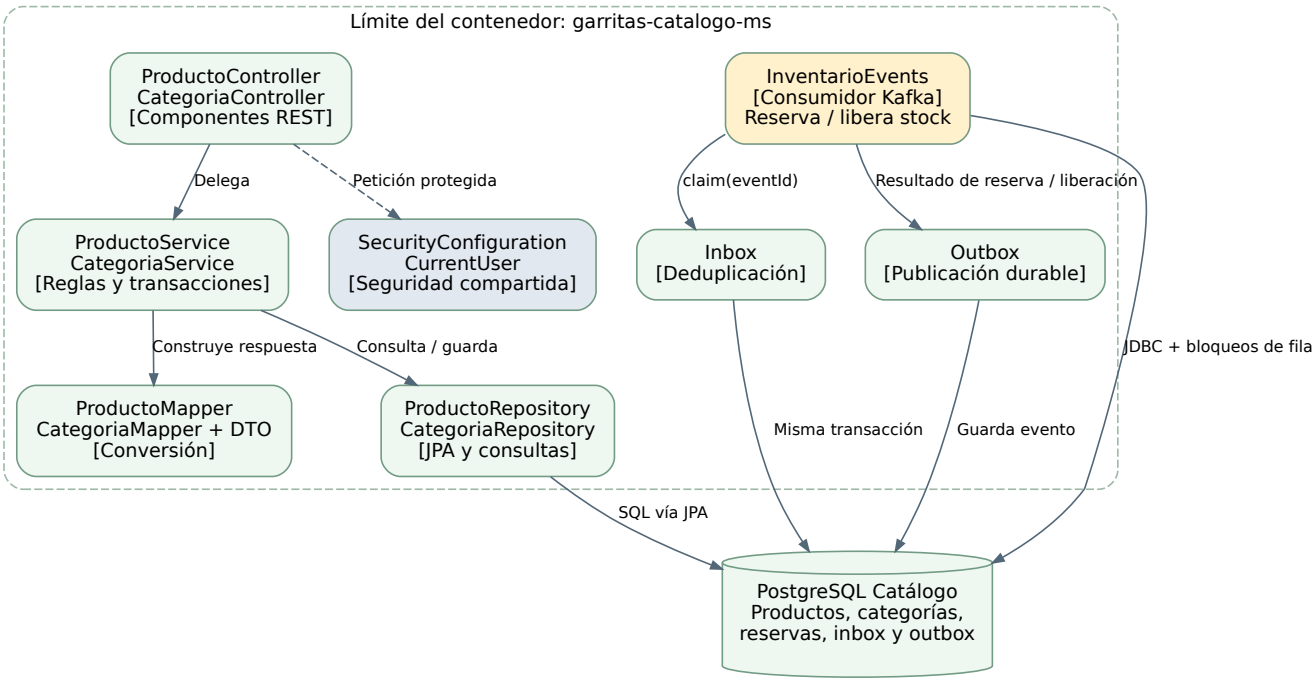
Reglas que debes poder explicar

Se rechazan cantidades inválidas y stock insuficiente. Los IDs de categorías deben existir. La búsqueda admite filtros y paginación. La eliminación de un producto es **baja lógica**: lo deja inactivo para conservar historial y reservas. La reserva Kafka bloquea filas en orden y verifica todos los productos antes de descontar stock.

Qué mostrar en Angular	Qué código lo respalda
Productos: crear y editar	ProductoController: endpoints de productos → ProductoService.crear(): alta de productos
Dos categorías para el mismo producto	Producto: relación N:M con categorías y Migración producto_categoria
Filtros y paginación	ProductoRepository.buscar(): filtros y paginación
Desactivar producto	ProductoService.eliminar(): baja lógica
Pago rechazado y stock recuperado	InventarioEvents.liberar(): compensación

> El precio que envía o muestra el navegador no se acepta como verdad para crear la venta. Venta obtiene el precio real desde Catálogo y la reserva vuelve a verificarlo.

6. C3: componentes de Catálogo



C4 · Nivel 3 — Componentes dentro de Catálogo

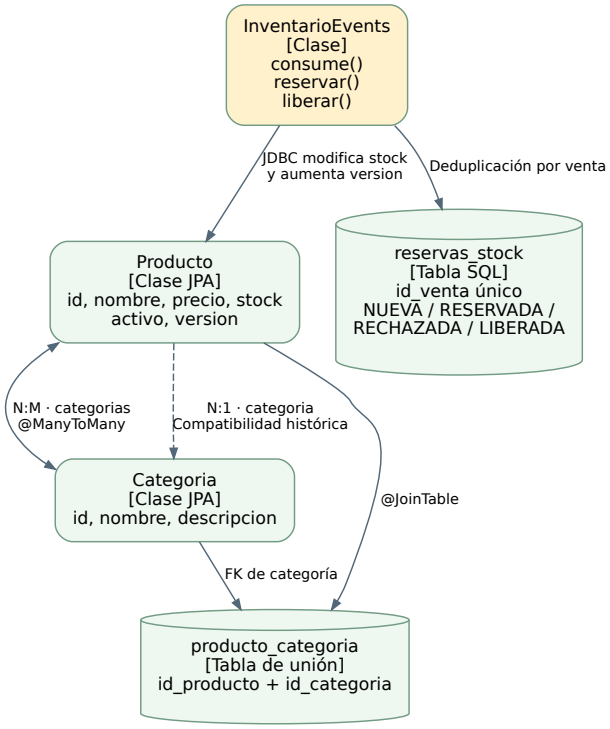
Flujo REST: Controller recibe la petición y valida el DTO; Service aplica reglas; Repository accede a PostgreSQL; Mapper prepara el DTO de salida.

Flujo Kafka: InventarioEvents recibe un contrato de evento, reclama su eventId en Inbox y reserva o libera stock mediante JDBC. Guarda el resultado en Outbox dentro de la misma transacción local. El publicador de Outbox lo envía después a Kafka.

ProductoService.crear(): alta de productos · línea 45

```
45    public ProductoResponse crear(ProductoRequest request) {
46      Producto producto = productoMapper.toEntity(request);
47      asignarCategoriasDesdeRequest(producto, request);
48      ProductoResponse response = productoMapper.toResponse(productoRepository.save(producto));
49      meterRegistry.counter("garritas_productos_creados_total").increment();
50      return response;
51    }
52
53    public ProductoResponse actualizar(Long id, ProductoRequest request) {
```

7. C4: clases y muchos a muchos de Catálogo



C4 · Nivel 4 — Código: clases y persistencia de Catálogo

Muchos a muchos que puedes demostrar

Un champú puede pertenecer a **Higiene** y **Cuidado**. A su vez, Higiene agrupa champú, cepillos y otros productos. La tabla `producto_categoria` guarda cada par de IDs.

Producto: relación N:M con categorías · línea 44

```
44    @ManyToMany(fetch = FetchType.LAZY)
45    @JoinTable(
46        name = "producto_categoria",
47        joinColumns = @JoinColumn(name = "id_producto"),
48        inverseJoinColumns = @JoinColumn(name = "id_categoria"))
49    private Set<Categoria> categorias = new HashSet<>();
50  }
```

Existe además el campo antiguo `categoria` con @ManyToOne para compatibilidad. No debes usarlo para explicar toda la relación actual: las múltiples categorías viven en `categorias` y en la tabla de unión.

Concurrencia e inventario

`@Version` protege actualizaciones JPA. Las operaciones JDBC de reserva/liberación aumentan también `version`. `reservas_stock.id_venta` identifica el efecto de negocio, incluso si llega otro eventId para la misma venta.

8. Qué administra Usuario

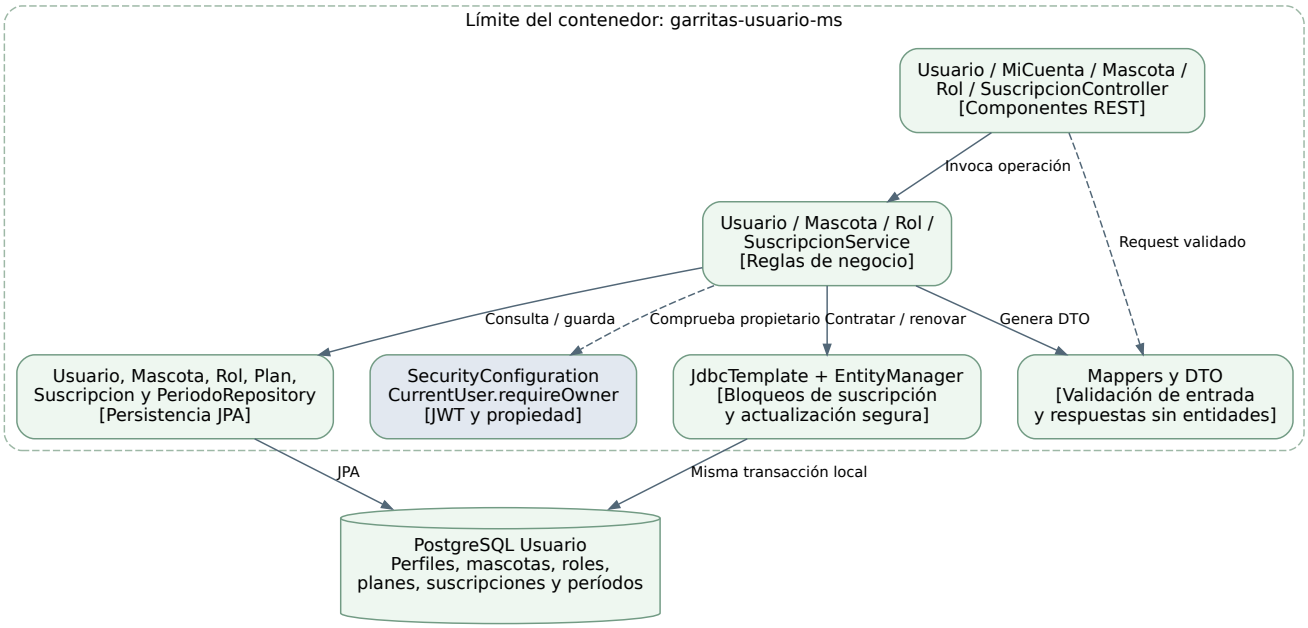
Responsabilidades

Capacidad	Qué conserva	Regla importante
Perfil	Nombre, correo, teléfono, dirección, DNI y subject.	El perfil propio se identifica con el sub del JWT; guardar comprueba persistencia.
Mascotas	Nombre, especie, raza, edad y DIAS/MESES.	Pertenecen a un usuario; se controla propiedad.
Roles internos	Clasificaciones como VENTAS o SECRETARIA.	No conceden permisos ADMIN en el JWT.
Planes	Nombre, precio, duración y estado.	Se desactivan sin borrar el historial contratado.
Suscripciones	Usuario, plan, estado y vigencia.	No duplicar una suscripción activa del mismo plan.
Períodos	ALTA o RENOVACION, precio y fechas acordados.	La misma clave de reintento no agrega otro período.

Teléfono opcional: si se informa, celular peruano de nueve dígitos y empieza con 9. DNI opcional: ocho dígitos. La edad de mascota se expresa como entero con unidad explícita.

> Keycloak guarda y verifica las credenciales de acceso. La tabla usuarios de este microservicio guarda el perfil del negocio; no contiene un campo de contraseña de login.

9. C3: componentes de Usuario

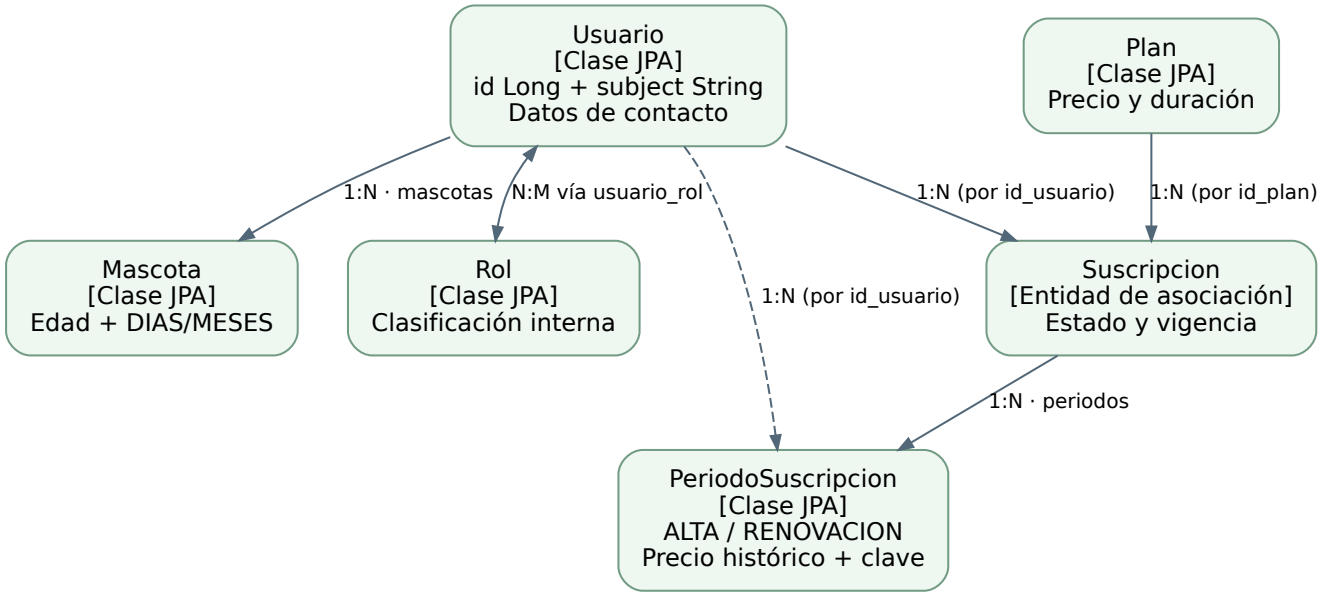


C4 · Nivel 3 — Componentes dentro de Usuario

MiCuentaController expone `/usuarios/me`: usa el token para operar sobre el perfil propio. UsuarioController permite el mantenimiento administrativo. MascotaService comprueba que la mascota corresponde al usuario. RolService valida el catálogo de clasificaciones.

SuscripcionService administra planes y el ciclo de contratación. Sus repositorios conservan cabecera e historial. Los bloqueos PostgreSQL serializan operaciones del usuario y la clave de idempotencia. Todo esto es una transacción local de Usuario, no una transacción distribuida entre bases.

10. C4: clases y relaciones de Usuario



C4 · Nivel 4 — Código: asociaciones de Usuario

Tres relaciones que puedes señalar

- **Usuario 1:N Mascota:** una familia registra varias mascotas; cada mascota tiene su propietario.
- **Usuario N:M Rol:** varios roles por usuario y varios usuarios por rol; unión `usuario_rol`.
- **Usuario N:M Plan mediante Suscripcion:** la asociación tiene estado, fechas y períodos; por eso es una entidad de negocio y no solamente @ManyToMany.

Usuario: relación N:M con Rol · línea 50

```
50    @ManyToMany(fetch = FetchType.LAZY)
51    @JoinTable(
52        name = "usuario_rol",
53        joinColumns = @JoinColumn(name = "id_usuario"),
54        inverseJoinColumns = @JoinColumn(name = "id_rol"))
55    private Set<Rol> roles = new HashSet<>();
56  }
```

Suscripcion: entidad de asociación Usuario–Plan · línea 18

```
18    @ManyToOne(fetch = FetchType.LAZY, optional = false)
19    @JoinColumn(name = "id_usuario")
20    private Usuario usuario;
21
22    @ManyToOne(fetch = FetchType.LAZY, optional = false)
23    @JoinColumn(name = "id_plan")
24    private Plan plan;
25
26    @Column(name = "nombre_plan", nullable = false, length = 100)
```

Los IDs de estas asociaciones pertenecen a la misma base de Usuario. Una referencia entre microservicios, como el id de mascota que guarda Citas, no se convierte en una FK entre bases independientes.

11. Dónde están los CRUD y reglas

CRUD significa crear, consultar, actualizar y eliminar. En este dominio algunas eliminaciones son desactivaciones; las suscripciones usan operaciones de negocio, no una edición arbitraria de todos sus campos.

Recurso y ruta por Gateway	Operaciones	Código
/api/v1/productos	GET lista/detalle; POST alta; PUT /{id}; DELETE /{id} desactiva.	ProductoController: endpoints de productos
/api/v1/categorias	GET lista/detalle; POST; PUT /{id}; DELETE /{id}.	CategoriaController: CRUD de categorías
/api/v1/usuarios	Administración: GET, POST, PUT /{id}, DELETE /{id} con restricciones de historial.	UsuarioController: gestión administrativa de usuarios
/api/v1/usuarios/me	GET y PUT del perfil autenticado.	MiCuentaController: perfil y mascotas propias
/api/v1/usuarios/me/mascotas	GET, POST; GET/PUT/DELETE /{id}, con propiedad.	MiCuentaController: perfil y mascotas propias
/api/v1/usuarios/{id}/mascotas	Mantenimiento de mascotas del usuario autorizado.	MascotaController: mascotas por usuario
/api/v1/roles o /api/v1/usuarios/roles	CRUD administrativo del catálogo interno.	RolController: catálogo de roles internos
/api/v1/usuarios/{id}/roles/{idRol}	POST asigna; DELETE retira la asociación.	UsuarioController.asignarRol(): asociación usuario–rol
/api/v1/planes	GET público; POST, PUT /{id} y DELETE /{id} administrativos.	SuscripcionController: planes y ciclo de suscripciones
/api/v1/suscripciones	GET y POST contratar; POST /{id}/renovar o /cancelar.	SuscripcionController: planes y ciclo de suscripciones

Cómo encontrar cualquier clase

Abre la raíz del proyecto en VS Code. Pulsa **Ctrl+P**, pega la ruta completa del mapa final y presiona Enter. Con **Ctrl+G** salta a la línea indicada. Los enlaces azules abren la misma ubicación en GitHub.

12. Eureka, Gateway y comunicación resiliente

Eureka: la agenda de direcciones

Cada servicio registra nombre, instancia y puerto, y mantiene su presencia con renovaciones. Eureka permite descubrir dónde está un servicio. No almacena productos ni transporta los eventos de Kafka. La detección de una caída no es instantánea.

Configuración: Configuración de Eureka DEV; registro de Registro de Catálogo en Eureka y Registro de Usuario en Eureka.

Gateway: la entrada de las APIs

Angular llama a `http://localhost:18080/api/v1`. Gateway aplica seguridad y enruta `/productos` a `lb://garritas-catalogo-ms` y `/usuarios` a `lb://garritas-usuario-ms`. El prefijo lb indica resolución y balanceo por nombre lógico.

Rutas reales: Rutas del Gateway hacia Catálogo y Usuario; dirección que usa Angular: Angular: dirección de la API del Gateway.

Feign y resiliencia: S6

Venta necesita consultar precio y disponibilidad antes de aceptar una orden. Feign expresa esa consulta como una interfaz Java con nombre lógico. El timeout limita una espera; el Circuit Breaker evita seguir insistiendo sobre una dependencia que falla; su fallback devuelve una respuesta controlada, sin inventar stock ni precio.

ProductoClient: Feign por nombre lógico · línea 7

```
 7  @FeignClient(name = "garritas-catalogo-ms")
 8  public interface ProductoClient {
 9    record Producto(Long id, String nombre, BigDecimal precio, Integer stock, Boolean activo) {}
10
11    @GetMapping("/api/v1/productos/{id}")
12    Producto obtener(@PathVariable Long id);
13  }
```

Consulta protegida: CatalogoConsulta: Circuit Breaker y respuesta controlada; límites: Timeouts Feign y configuración de los circuitos. En DEV hay connectTimeout de 2 segundos y readTimeout de 3 segundos; el circuito de Catálogo tiene ventana 10, mínimo 5 llamadas, umbral 50% y apertura de 10 segundos.

> No toda llamada interna usa Feign. Citas utiliza ServiceClient con descubrimiento y tiempos acotados para consultar Usuario. No dibujes un Circuit Breaker que no exista en esa llamada.

13. Keycloak, JWT y perfil propio

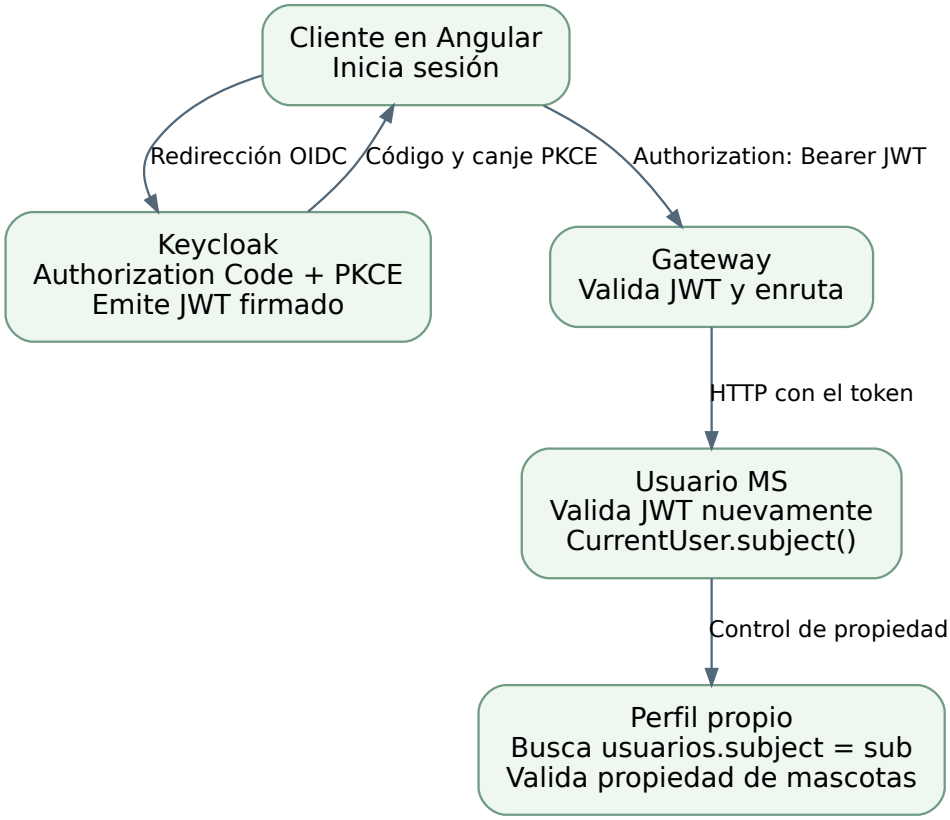


Diagrama dinámico — Identidad y protección del perfil

Autenticación: Keycloak identifica al usuario. **Autorización:** Gateway y cada servicio deciden qué puede hacer según el JWT y el recurso.

El token contiene `sub` (identidad de tipo String) y `realm_access.roles`. La conversión añade el prefijo ROLE_ para Spring Security. El id local de la fila de usuario sigue siendo Long; no debe confundirse con sub.

El servicio valida firma, emisor y audiencia configurada. Las claves públicas se consultan en el endpoint JWKS; la clave privada de firma permanece en Keycloak. Para el perfil propio se compara subject con la identidad autenticada.

CurrentUser.requireOwner(): propiedad de recursos · línea 18

```
18    public static void requireOwner(String owner) {
19      if (!admin() && !subject().equals(owner))
20        throw new AccessDeniedException("El recurso pertenece a otro cliente");
21    }
22  }
```

Ubicaciones: SecurityConfiguration: protección de rutas, Conversión de roles reales del JWT, Emisor, claves públicas y audiencia del JWT y Auth de Angular: PKCE y sesión.

> El cliente Angular usa Authorization Code + PKCE y administra tokens en memoria; no guarda access_token en localStorage. Las guardas o botones ocultos no sustituyen la autorización del backend. Los roles internos de PostgreSQL no cambian el rol firmado por Keycloak.

14. Kafka, Outbox e Inbox

Kafka: comunicar hechos de negocio

Un productor publica un evento y cada grupo consumidor procesa los eventos que le corresponden. Garritas utiliza `garritas.ventas.v1`, con tres particiones y un grupo independiente para Venta, Catálogo y Pago. La clave de publicación es la venta, para distribuir sus eventos de forma coherente por partición.

REST/Feign: el llamador espera una respuesta ahora. **Kafka:** se informa un hecho y el resultado final puede conocerse después. Hay desacople temporal y consistencia eventual; deben manejarse duplicados.

Pieza	Qué problema resuelve	Código
SagaEvent	Contrato con eventId, tipo, ventaId, cliente, total, items y traceId.	SagaEvent: contrato del evento
Outbox	Guarda el evento junto con el cambio de base; lo publica después.	Outbox.append(): cambio y evento en una transacción y Outbox.publish(): publicación posterior a Kafka
Inbox	Deduplica por eventId dentro de la transacción del consumidor.	Inbox.claim(): deduplicación por eventId
reserva por venta	Evita repetir el efecto aunque llegue otro eventId para la misma venta.	InventarioEvents.reservar(): reserva e idempotencia
Reintentos y DLT	El manejador aplica reintentos finitos y deriva errores no resueltos a .DLT.	Reintentos y topic DLT

La entrega puede repetirse: si el proceso cae después del ACK de Kafka y antes de marcar publicado el outbox, el evento podría volver a salir. Por eso no prometemos “exactamente una vez” global; construimos efectos idempotentes.

Un mensaje que acaba en DLT requiere diagnóstico y reprocesamiento controlado; no se recupera por arte de magia. Usuario no implementa un consumidor de esta Saga. Su idempotencia de suscripciones se basa en la clave de petición y los períodos guardados en su propia base.

15. Saga: fallos y compensación

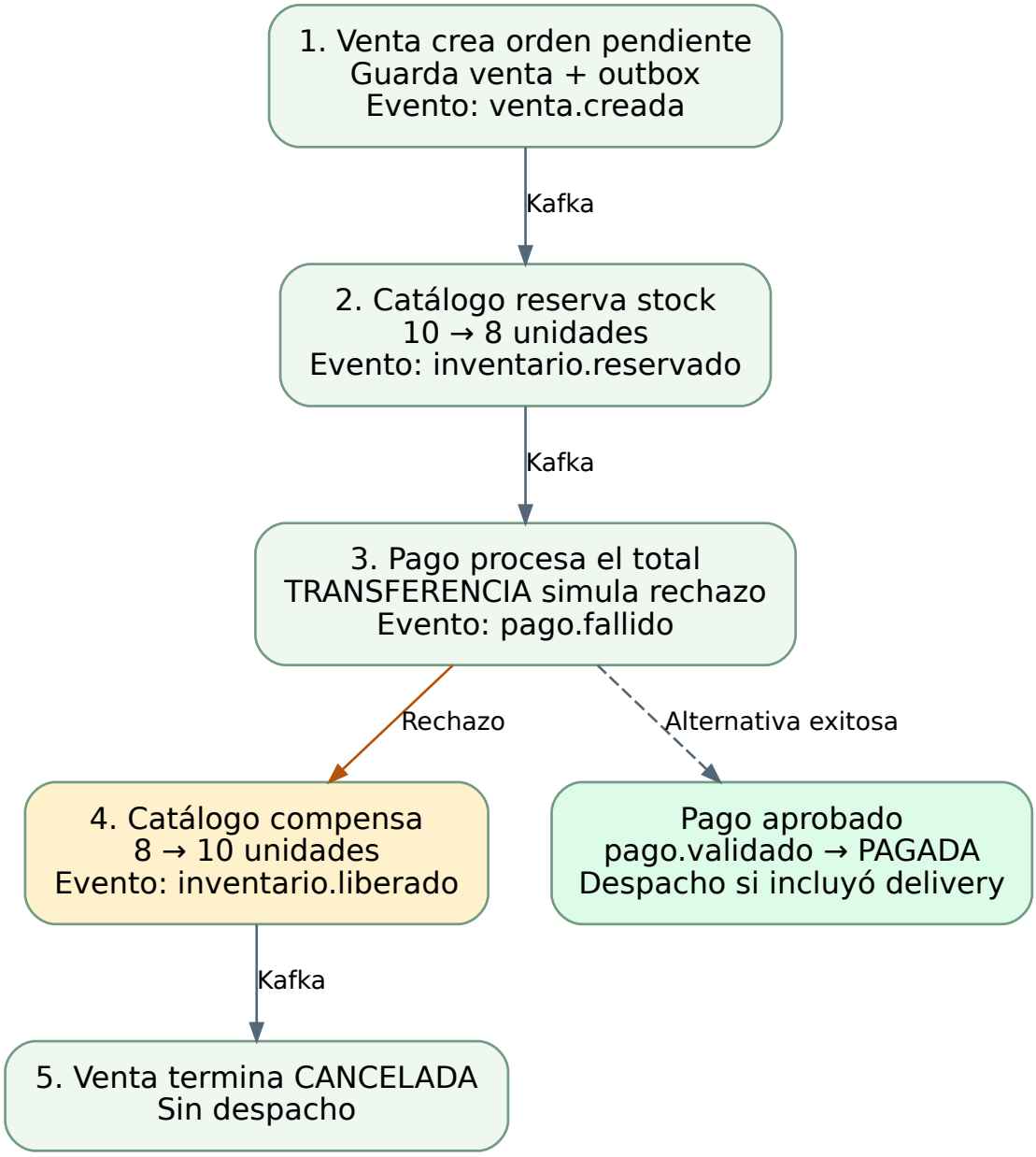


Diagrama dinámico — Saga y compensación de inventario

Compensación real

Es una operación nueva que revierte un efecto de negocio ya confirmado. Catálogo pasó de stock 10 a 8 al reservar dos unidades. Ante pago.fallido, `liberar()` recupera las cantidades de la reserva original, pasa de 8 a 10 y marca la reserva LIBERADA. Si el evento vuelve a llegar, no debe liberar nuevamente.

InventarioEvents.liberar(): compensación · línea 76

```
 76    private void liberar(SagaEvent e) {
 77      var rows =
 78          jdbc.queryForList(
 79              "SELECT estado,payload FROM reservas_stock WHERE id_venta=? FOR UPDATE",
e.ventaId());
 80      if (rows.isEmpty()) throw new IllegalStateException("Reserva inexistente: " + e.ventaId());
 81      if (!rows.getFirst().get("estado").equals("RESERVADA")) return;
 82      var original = json.readValue(rows.getFirst().get("payload").toString(), SagaEvent.class);
 83      for (var item :
 84
original.items().stream().sorted(Comparator.comparing(SagaEvent.Item::productoId)).toList())
 85        jdbc.update(
 86            "UPDATE productos SET stock=stock+?,version=version+1 WHERE id=?",
 87            item.cantidad(),
 88            item.productoId());
 89      jdbc.update("UPDATE reservas_stock SET estado='LIBERADA' WHERE id_venta=?", e.ventaId());
 90      outbox.append(original.next("inventario.liberado"));
 91    }
 92  }
```

Situación	Qué hace Garritas	Clasificación
Falla una validación previa	No se crea la venta.	Validación / resiliencia
Falla la reserva asíncrona	Venta existente termina cancelada sin pago.	Rechazo del proceso
Pago rechaza tras reservar	Catálogo devuelve stock; Venta COMPENSANDO → CANCELADA.	Compensación
Pago/Kafka temporalmente detenido	El proceso puede continuar tras recuperarse; revisar reintentos y DLT ante errores persistentes.	Recuperación
Evento repetido	No se repite el efecto.	Idempotencia
Error antes del commit de una base	Se deshacen cambios de esa transacción.	Rollback local

> No existe compensación universal de todos los módulos ni una transacción ACID que abarque sus bases. El pago de Garritas es simulado. Cancelar una suscripción es una transición local; no equivale a un reembolso bancario.

16. Credenciales, claves e identificadores

“Claves” puede referirse a contraseñas, identificadores o claves de seguridad. Aquí se separan para no confundirlos.

Credenciales de demostración de la configuración DEV

Acceso	Usuario / contraseña inicial	Dónde está
Angular cliente	cliente / Cliente123!	Usuario cliente de demostración en Keycloak
Angular administrador	admin-store / Admin123!	Usuario administrador de la tienda
Keycloak, consola de administración	admin / local-admin-change-me	Contraseña de administración inicial de Keycloak
Grafana	admin / local-grafana	Contraseña inicial de Grafana DEV
PostgreSQL Catálogo	garritas / garritas · DB garritas_catalogo_db	PostgreSQL de Catálogo: credenciales DEV
PostgreSQL Usuario	garritas / garritas · DB garritas_usuario_db	PostgreSQL de Usuario: credenciales DEV

Son valores de desarrollo publicados en el repositorio. Si cambiaste la contraseña en una instalación o definiste la variable de entorno correspondiente, prevalece ese valor. Editar el archivo de importación del realm no restablece automáticamente la contraseña de un usuario existente.

Claves de firma y claves de idempotencia

Término	Significado y ubicación
Clave privada JWT	Keycloak la administra. No se pega una clave privada en Angular ni en Usuario MS.
Claves públicas JWKS	http://localhost:18180/realms/garritas/protocol/openid-connect/certs · Emisor, claves públicas y audiencia del JWT
sub	Identificador del usuario autenticado; se guarda como subject. UsuarioService.miUsuario(): identidad y perfil
Idempotency-Key	UUID enviado para reintentar una operación sin duplicarla. SuscripcionController: planes y ciclo de suscripciones
eventId	Identificador de un evento; Inbox lo usa para deduplicar. Inbox.claim(): deduplicación por eventId
idVenta	Clave de negocio de la reserva; protege también contra duplicados semánticos. InventarioEvents.reservar(): reserva e idempotencia

> La SPA es un cliente público (publicClient=true): no tiene client_secret. Su protección de login se basa en PKCE.

17. Cómo exponerlo desde Angular

Guion para Fio: tres minutos, con datos preparados

Tiempo	Acción visible	Frase para explicar
0:00–0:40	Administrador → Productos. Muestra un producto con dos categorías y usa un filtro.	“Catálogo mantiene productos y la relación muchos a muchos Producto–Categoría”.
0:40–1:15	Cliente → Mi cuenta y mascotas. Guarda un dato y recarga; muestra edad con unidad.	“El perfil se vincula al sub del token; la mascota pertenece a este cliente”.
1:15–1:50	Planes y suscripciones. Muestra un contrato y sus períodos; utiliza un cliente preparado.	“Usuario registra una operación con estado e historial. Un reintento con la misma clave no agrega otro período”.
1:50–3:00	Anota stock; compra con TRANSFERENCIA; espera cancelación y comprueba stock recuperado. Apoya con logs/eventos.	“La reserva se confirmó, el pago falló y Catálogo compensó liberando las unidades”.

Pestañas listas antes de empezar

Angular en una ventana de cliente y otra de administrador; Kafka UI en :18085; Grafana en :13000; mapa de código abierto. No uses la cuenta administradora para comprar. Preparar datos evita dedicar la demo a escribir formularios largos.

Qué observar en las herramientas

En Kafka UI selecciona el topic de ventas y localiza venta.creada, inventario.reservado, pago.fallido e inventario.liberado por ventaId. En Grafana, Explore → Loki permite buscar el traceId y comparar servicios. Para disponibilidad de Keycloak usa Prometheus: `up{job="keycloak"}`; esto demuestra scrape correcto, no por sí solo un login exitoso.

> Presionar Renovar dos veces con claves nuevas son dos operaciones distintas. Para demostrar idempotencia, hay que repetir la misma petición con el mismo propietario y la misma Idempotency-Key.

18. Preguntas y respuestas rápidas

¿Qué diferencia hay entre Catálogo y Usuario?

Catálogo administra lo que se vende y su inventario; Usuario administra perfiles, mascotas y el ciclo de suscripciones.

¿Dónde está mi muchos a muchos?

En Producto.categorias → producto_categoria y Usuario.roles → usuario_rol. Usuario–Plan se representa mediante Suscripcion porque la asociación tiene datos de negocio.

¿Usuario autentica con contraseña?

No. Keycloak autentica; Usuario conserva el perfil asociado al sub y valida el JWT para sus rutas protegidas.

¿Para qué Eureka si ya existe Gateway?

Gateway es la entrada y enruta; Eureka permite descubrir las instancias a las que enruta y a las que otros servicios consultan.

¿Kafka reemplaza Feign?

No. Feign consulta datos que se necesitan inmediatamente; Kafka desacopla pasos que pueden completarse después.

¿Qué compensación implementaste?

Liberar stock reservado cuando el pago es rechazado. El efecto confirmado se revierte mediante otra transacción y otro evento.

¿Cancelar un estado basta para compensar?

No. Hay que deshacer el efecto correspondiente: por ejemplo, recuperar las unidades reservadas.

¿Por qué el duplicado no libera dos veces?

Inbox detecta el eventId y la reserva por venta solo se libera si sigue RESERVADA. Una reserva LIBERADA no vuelve a sumar stock.

¿Todo queda consistente de inmediato?

No. Hay estados intermedios y consistencia eventual. Se recupera el proceso y se diagnostican mensajes fallidos con logs, métricas y DLT.

¿Qué demuestran los tests existentes?

La revisión cf3cede documenta 30 pruebas Java y 15 Angular más verificaciones de flujos y fallos. Esta entrega es documental: no vuelve a ejecutar ni certifica tu instalación Windows.

19. Mapa completo del código

Rutas relativas a la raíz `Garritas-actualizado`. En Windows puedes usar Ctrl+P y pegar la ruta con las barras tal como aparece. La línea corresponde al commit `cf3cede`.

Qué buscar	Archivo exacto	Línea
ProductoController: endpoints de productos	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/controller/ProductoController.java	16
ProductoService.crear(): alta de productos	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/service/ProductoService.java	45
ProductoService.actualizar(): edición y categorías	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/service/ProductoService.java	53
ProductoService.eliminar(): baja lógica	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/service/ProductoService.java	64
ProductoRepository.buscar(): filtros y paginación	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/repository/ProductoRepository.java	30
Producto: relación N:M con categorías	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/entity/Producto.java	44
ProductoMapper: entidad a respuesta	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/mapper/ProductoMapper.java	30
CategoriaController: CRUD de categorías	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/controller/CategoriaController.java	16
CategoriaService: reglas y persistencia	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/service/CategoriaService.java	16
InventarioEvents.consume(): entrada de Kafka	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/messaging/InventarioEvents.java	25
InventarioEvents.reservar(): reserva e idempotencia	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/messaging/InventarioEvents.java	37
Bloqueo de filas antes de cambiar stock	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/messaging/InventarioEvents.java	52
InventarioEvents.liberar(): compensación	services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/messaging/InventarioEvents.java	76
UsuarioController: gestión administrativa de usuarios	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/UsuarioController.java	16
MiCuentaController: perfil y mascotas propias	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/MiCuentaController.java	14
UsuarioService.guardarMe(): persistencia por sub	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/UsuarioService.java	95
UsuarioService.miUsuario(): identidad y perfil	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/UsuarioService.java	128
Usuario: relación N:M con Rol	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/entity/Usuario.java	50

Qué buscar	Archivo exacto	Línea
Usuario: relación 1:N con Mascota	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/entity/Usuario.java	42
RolController: catálogo de roles internos	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/RolController.java	15
RolService.eliminar(): protección de roles usados	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/RolService.java	50
UsuarioController.asignarRol(): asociación usuario–rol	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/UsuarioController.java	62
MascotaController: mascotas por usuario	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/MascotaController.java	15
MascotaService.crear(): alta y propiedad	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/MascotaService.java	29
UsuarioRequest: validación del celular y DNI	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/dto/UsuarioRequest.java	26
MascotaRequest: edad y unidad explícita	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/dto/MascotaRequest.java	25
SuscripcionController: planes y ciclo de suscripciones	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/SuscripcionController.java	14
SuscripcionService.contratar(): proceso transaccional	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/SuscripcionService.java	64
SuscripcionService.renovar(): período e idempotencia	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/SuscripcionService.java	98
SuscripcionService.cancelar(): transición local	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/SuscripcionService.java	134
Bloqueos de suscripción y clave de reintento	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/SuscripcionService.java	142
Suscripcion: entidad de asociación Usuario–Plan	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/entity/Suscripcion.java	18
PeriodoSuscripcion: clave y precio histórico	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/entity/PeriodoSuscripcion.java	28
UsuarioRepository: búsqueda por rol y DNI/nombre	services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/repository/UsuarioRepository.java	23
SecurityConfiguration: protección de rutas	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/SecurityConfiguration.java	11
Conversión de roles reales del JWT	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/SecurityConfiguration.java	26
CurrentUser.requireOwner(): propiedad de recursos	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/CurrentUser.java	18
Outbox.append(): cambio y evento en una transacción	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/Outbox.java	35
Outbox.publish(): publicación posterior a Kafka	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/Outbox.java	48

Qué buscar	Archivo exacto	Línea
Inbox.claim(): deduplicación por eventId	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/Inbox.java	13
SagaEvent: contrato del evento	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/SagaEvent.java	9
MessagingConfiguration: topic y particiones	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/MessagingConfiguration.java	31
Reintentos y topic DLT	infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/MessagingConfiguration.java	41
VentaService.crear(): validación, total y outbox	services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/service/VentaService.java	49
VentaEvents: estados de Saga y despacho pagado	services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/messaging/VentaEvents.java	22
ProductoClient: Feign por nombre lógico	services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/client/ProductoClient.java	7
CatalogoConsulta: Circuit Breaker y respuesta controlada	services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/client/CatalogoConsulta.java	18
PagoEvents: pago simulado y rechazo	services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/messaging/PagoEvents.java	33
Rutas del Gateway hacia Catálogo y Usuario	infra/garritas-config/config-repo/garritas-gateway-dev.yml	26
Configuración de Eureka DEV	infra/garritas-config/config-repo/garritas-eureka-dev.yml	2
Registro de Catálogo en Eureka	infra/garritas-config/config-repo/garritas-catalogo-ms-dev.yml	69
Registro de Usuario en Eureka	infra/garritas-config/config-repo/garritas-usuario-ms-dev.yml	45
Emisor, claves públicas y audiencia del JWT	infra/garritas-config/config-repo/garritas-usuario-ms-dev.yml	30
Timeouts Feign y configuración de los circuitos	infra/garritas-config/config-repo/garritas-venta-ms-dev.yml	60
Kafka: direcciones internas y externas	kafka/compose-dev.yml	16
Grupo consumidor propio de Catálogo	infra/garritas-config/config-repo/garritas-catalogo-ms-dev.yml	47
PostgreSQL de Catálogo: credenciales DEV	services/garritas-catalogo-ms/compose-dev.yml	11
PostgreSQL de Usuario: credenciales DEV	services/garritas-usuario-ms/compose-dev.yml	11
Usuario cliente de demostración en Keycloak	keycloak/garritas-realm.json	47
Usuario administrador de la tienda	keycloak/garritas-realm.json	65
Contraseña de administración inicial de Keycloak	keycloak/compose-dev.yml	32
Contraseña inicial de Grafana DEV	obs/compose-dev.yml	46
Auth de Angular: PKCE y sesión	clients/garritas-frontend/src/app/core/auth.ts	29

Qué buscar	Archivo exacto	Línea
Angular: dirección de la API del Gateway	clients/garritas-frontend/public/config.json	1
Angular: formulario de producto y categorías	clients/garritas-frontend/src/app/features/admin/productos.component.ts	149
Angular: guardar y volver a consultar el perfil	clients/garritas-frontend/src/app/features/cuenta/cuenta.component.ts	61
Angular: asignación de roles internos	clients/garritas-frontend/src/app/features/admin/usuarios-roles.component.ts	16
Angular: contratación de planes	clients/garritas-frontend/src/app/features/planes/planes.component.ts	118
Migración producto_categoria	services/garritas-catalogo-ms/src/main/resources/db/migration/V5__producto_categorias_many_to_many.sql	1
Migración rol y usuario_rol	services/garritas-usuario-ms/src/main/resources/db/migration/V5__create_roles_tables.sql	1
Migración de suscripciones y períodos	services/garritas-usuario-ms/src/main/resources/db/migration/V4__suscripciones_periodos.sql	2